



PhenoRx

Pharmacogenomic Adverse Drug Reaction Risk Assessment

A technical and methodological overview of a clinical decision support prototype that combines a machine-learning classifier, pharmacogenomic rule engines, and a large language model to estimate adverse drug reaction risk before a medicine is administered.

Written for medical students and clinicians with no machine-learning background. It explains what the system does, how each component works, and — importantly — how well it actually performs.

This is a teaching and research prototype, not a clinical tool.

It is trained on synthetic data, has never been externally validated, and its measured accuracy is below that of a trivial baseline. It must not be used to make decisions about any real patient. Section 6 explains the evidence for this statement in full.

Document	Technical & Methodological Overview
System version	2.1.0
Model artifact	<code>models/adr_model.pkl</code> (17.9 MB)
Audience	Medical students, clinical pharmacology trainees

Contents

- 1 The clinical problem** Why ADR prediction matters

- 2 What the system does** The clinical workflow, page by page

- 3 Architecture** Three independent layers

- 4 Layer 1 — The machine-learning model** Algorithm, features, training

- 5 Layer 2 — Pharmacogenomic rule engines** CYP, transporters, HLA

- 6 Model performance — the honest numbers** What it scores, and why

- 7 Layer 3 — The AI narrative** Large language model integration

- 8 Safety, limitations and governance** What would be needed clinically

- 9 Glossary** Machine-learning terms in clinical language

How to read this document

Sections 1–3 describe the system and need no technical background. Sections 4–7 go into the methods. **Section 6 is the most important section** — it reports the measured performance, which is poor, and explains precisely why. If you read only one part, read that one: it is the part that teaches you how to interrogate any clinical AI tool you are shown in future.

1 The clinical problem

An adverse drug reaction (ADR) is harm caused by a medicine given at a normal dose for a normal indication. ADRs are a substantial cause of hospital admission and inpatient morbidity, and a meaningful proportion are considered preventable.

The clinical difficulty is that risk is not uniform. Two patients given the same drug at the same dose can have very different outcomes, because risk depends on the interaction of:

- **Organ function** — chiefly renal clearance (eGFR) and hepatic metabolism, which determine how quickly a drug is eliminated.
- **Genetics** — inherited variation in metabolising enzymes and immune receptors, which can multiply exposure or predispose to severe immune reactions.
- **Co-medication** — competing substrates, enzyme inhibition or induction, and additive toxicity across a regimen.
- **Comorbidity and physiology** — age, body composition, cardiac disease, electrolyte state.

A clinician integrates these mentally. The premise of this project is that the same integration can be performed computationally, before the first dose, to flag patients who warrant a different agent, a lower dose, or closer monitoring.

The premise is reasonable; the execution here is a prototype

The idea above is sound and is an active area of real research. This particular implementation does not achieve it — see Section 6. The value of this document is in showing how such a system is constructed and how to evaluate whether it works.

Why "phenotype" plus "Rx"

The name reflects the intended approach: combine the patient's observable clinical phenotype (laboratory values, comorbidities, physiology) with their pharmacogenomic phenotype (metaboliser status) to individualise the prescription.

2 What the system does

The application is a web tool organised around a single clinical assessment. A user works through five screens in sequence.

1 • Patient details

Demographics: age, sex, ethnicity, height and weight. Body mass index is calculated automatically, and age and sex select the laboratory reference intervals used on every subsequent screen.

2 • Clinical assessment

The main data-entry form: renal and hepatic panels, full blood count and differential, vital signs, comorbidities, the drug regimen, and pharmacogenomic results. Values are validated against the same numeric bounds the server enforces, so an implausible entry is flagged as it is typed.

~60 fields, fillable in any order; a demo patient can populate all of them

3 • Report

The model's numerical output and the AI-generated narrative are presented side by side: overall risk percentage, risk tier, predicted reaction classes, contributing factors, and pharmacogenomic profile. Exportable as PDF.

4 • Interactions and dosing

Pairwise interaction checking across the regimen with severity classification, plus renal-, hepatic- and CYP-adjusted dose suggestions with monitoring intervals.

5 • Counselling

A six-step structured medication education workflow, including teach-back comprehension scoring and signed acknowledgement from both patient and counsellor.

Risk tiers

The model returns a probability, which is mapped to one of four tiers. The thresholds are fixed constants in the code, not learned from data:

Risk tier thresholds, from `services/ml_service.py`

TIER	OVERALL ADR RISK	INTENDED INTERPRETATION
Critical	≥ 75%	Consider an alternative agent; specialist input
High	≥ 50%	Dose reduction and active monitoring
Moderate	≥ 25%	Standard dosing with scheduled review
Low	< 25%	Routine care

A point worth noticing

These cut-points are arbitrary round numbers. In a validated clinical tool, thresholds would be derived from the data — chosen to achieve a target sensitivity or to optimise net clinical benefit, then tested on a separate cohort. Fixed round numbers are a reasonable placeholder in a prototype, but they are an assumption, not a finding.

3 Architecture

Three independent layers contribute to the output. They are worth separating clearly, because they have very different reliability characteristics.

LAYER	WHAT IT IS	HOW IT CAN FAIL
1. ML classifier	A gradient-boosting model trained on 10,000 synthetic patient records	Silently wrong. Produces a confident-looking number regardless of whether it is correct. This is the weakest layer — see Section 6.
2. PGx rule engines	Deterministic rules encoding published gene–drug relationships	Transparent. Rules are explicit and auditable; failure means a rule is missing or out of date, which is inspectable.
3. LLM narrative	A large language model that writes the clinical prose	Fluent but unreliable. Can produce plausible, well-written statements that are incorrect. Degrades to fixed rule-based text when unavailable.

Technology stack

Backend	Python 3.13, Flask (application factory with blueprints)
Machine learning	scikit-learn — HistGradientBoostingClassifier
Frontend	React 19 with Vite, Tailwind CSS, single-page application
Language model	NVIDIA NIM (Nemotron), OpenAI-compatible API
Serving	Gunicorn, one worker with four threads, containerised
Persistence	In-memory only — no database

Why the three layers are kept separate

Separation means a failure in one layer does not silently corrupt the others. If the language model is unreachable, the system still returns the model's number and the pharmacogenomic findings, with deterministic text in place of the narrative. The converse also holds: a fluent AI narrative does not make the underlying risk number any more trustworthy.

4 Layer 1 — The machine-learning model

The algorithm

The model is a **histogram-based gradient boosting classifier** (`HistGradientBoostingClassifier` from scikit-learn). In clinical terms, it can be understood as follows.

A single *decision tree* is a flowchart of yes/no questions — "is eGFR below 30?", "is the patient over 70?" — ending in a prediction. One tree alone is crude. *Gradient boosting* builds hundreds of trees in sequence, where each new tree is trained specifically on the cases the existing trees got wrong. The final prediction is the accumulated vote of all the trees.

This family of model is a sensible default for tabular clinical data: it handles mixed numeric and categorical variables, tolerates missing values natively, and captures interactions between variables without being told to look for them.

Training hyperparameters, from `config/ml_config.yaml` defaults

PARAMETER	VALUE	MEANING
<code>max_iter</code>	400	Maximum number of boosting trees
<code>max_depth</code>	18	Maximum questions deep any one tree may go
<code>learning_rate</code>	0.07	How much each tree is allowed to correct its predecessors
<code>l2_regularization</code>	0.3	Penalty discouraging over-confident fitting
<code>random_state</code>	42	Fixed seed, so training is reproducible
train/test split	80 / 20	A fifth of data held back to test on

Input features

The model consumes **29 variables** per patient:

DOMAIN	VARIABLES
Demographics	age, sex, ethnicity, weight, height, BMI
Renal	creatinine, eGFR
Hepatic	AST/ALT, bilirubin, albumin
Haematology	haemoglobin, white cell count, platelet count
Vital signs	systolic and diastolic blood pressure, heart rate, temperature
Comorbidity	diabetes, liver disease, chronic kidney disease, cardiac disease, hypertension
Drug exposure	index drug dose, concomitant drug count, polypharmacy flag, days since start
Pharmacogenomic	CYP2C9 and CYP2D6 metaboliser status

Note the mismatch between form and model

The assessment form collects roughly 60 fields, including an extensive pharmacogenomic panel (six CYP enzymes, three transporters, three HLA alleles). The model itself uses only the 29 variables above, of which just **two** are pharmacogenomic. The remaining PGx data is used by the rule engines in Layer 2, not by the model. This is worth knowing: entering a full genomic panel does not make the risk *number* more informed than entering CYP2C9 and CYP2D6 alone.

Training data

Records	10,000
Origin	Synthetic — generated by <code>scripts/data_generator.py</code>
Outcome classes	10 distinct ADR types, including "No ADR"
Split	8,000 training / 2,000 held-out test

The data is synthetic, and this is the root limitation

No real patient contributed to this model. The records were produced by a script that encodes assumptions about how clinical variables relate to adverse outcomes. A model trained this way can only learn the rules its generator already contained — it cannot discover anything about real physiology, and good performance on such data would demonstrate nothing about real-world accuracy.

5 Layer 2 — Pharmacogenomic rule engines

Separate from the model, four deterministic engines encode published gene–drug relationships. These are ordinary conditional logic, not learned from data, which makes them transparent and auditable.

ENGINE	SCOPE
CYP engine	Cytochrome P450 metaboliser phenotypes. Poor metabolisers accumulate an active drug; ultra-rapid metabolisers may fail to respond, or over-produce an active metabolite from a prodrug.
Transporter engine	Membrane transporter variants affecting drug distribution and hepatic uptake — the mechanism underlying, for example, statin-associated muscle symptoms.
HLA engine	Human leukocyte antigen alleles associated with severe immune-mediated hypersensitivity, the class of reaction where pre-prescription genotyping has the clearest established clinical value.
Non-CYP engine	Other metabolising enzymes outside the P450 system.

Why this layer is more trustworthy than the model

A rule stating a relationship between a genotype and a drug can be traced to a published source, reviewed by a pharmacologist, and corrected when guidance changes. Its behaviour is identical every time and can be reasoned about without running it. The machine-learning model has none of these properties: it is a numerical function fitted to data, and its reasoning cannot be inspected directly.

The generalisable lesson

When you encounter a clinical AI product, ask which parts are learned and which are encoded rules. Rules can be checked against guidelines. Learned components can only be trusted to the extent that they have been validated on data resembling your patients.

6 Model performance — the honest numbers

This section reports what the model actually scores. The results are poor. They are presented in full because a clinical decision support tool that overstates its accuracy is more dangerous than one that has none.

6.1 Measured accuracy

The repository contains two stored performance reports, and the shipped model can be re-evaluated directly.

All three disagree:

Reported accuracy of the ADR classifier, by source

SOURCE	ACCURACY	WEIGHTED F1	TEST SET
<code>reports/experiment_metrics.json</code> Training-run holdout	45.0%	0.407	200
<code>reports/model_evaluation_report.md</code> Stored evaluation	11.0%	0.126	200
Shipped model, re-evaluated Against <code>data/clinical_data.csv</code>	15.5%	0.183	200

6.2 The number that matters most

15.5% MODEL ACCURACY Shipped artifact, measured	89.5% BASELINE ACCURACY Always answer "No ADR"	−74 pts MODEL VS BASELINE Worse than guessing
--	---	--

An accuracy figure is meaningless without the baseline it must beat. In the training data, **8,951 of 10,000 records (89.5%) are labelled "No ADR"**. A trivial rule that ignores every input and always answers "No ADR" would therefore be right 89.5% of the time.

The model scores far below that. **It is substantially worse than the simplest possible guess.**

This is the single most important fact in this document

A model performing below its majority-class baseline has learned nothing useful about the outcome. Any risk score it produces should be treated as uninformative. This is also why accuracy alone is a poor metric for rare outcomes — a point that generalises to every diagnostic study you will read.

6.3 Why the numbers are so poor

Three separate, identifiable causes:

Cause 1 — Severe class imbalance

The outcome distribution is extremely skewed: the commonest class has 8,951 examples, the rarest has 46. Gradient boosting minimises overall error, so with this distribution the lowest-error strategy is to predict the

majority class and largely ignore the rest. Mitigations exist — class weighting, resampling, or optimising a metric other than accuracy — and none has been applied here.

Cause 2 — The evaluation is measuring an impossible task

The model is trained on **10** outcome classes but evaluated against a file containing **18**, of which only 4 overlap. **14 of the classes in the test set are ones the model was never trained to predict** and therefore cannot possibly output.

Label space mismatch between training and evaluation

DATASET	CLASSES	TARGET COLUMN
Training (<code>adr_training_data.csv</code>)	10	<code>adr_type</code>
Evaluation (<code>clinical_data.csv</code>)	18	<code>adr_risk_label</code>
Shared between them	4	—

The two files also differ in shape — 30 columns versus 50. So the 11% and 15.5% figures do not measure how good the model is; they largely measure a mismatch between two incompatible datasets. The evaluation harness is broken independently of the model.

Cause 3 — Synthetic training data

Even a perfect score here would not indicate clinical utility, for the reason given in Section 4: the model can only recover the assumptions its data generator encoded.

6.4 Per-class breakdown

Accuracy hides the detail. In the stored per-class results, the majority of ADR classes have precision and recall of **exactly zero** — the model never once correctly identifies them. A representative extract:

Per-class performance, from `reports/experiment_metrics.json`

ADR CLASS	PRECISION	RECALL	F1	SUPPORT
No ADR	0.66	0.88	0.75	52
Cardiovascular Event (Arrhythmia)	0.54	0.60	0.57	35
Nephrotoxicity	0.47	0.50	0.48	16
Hepatotoxicity	0.36	0.50	0.42	8
QT Prolongation	0.35	0.33	0.34	21
Gastrointestinal (Nausea/Vomiting)	0.00	0.00	0.00	13
Elevated Liver Enzymes	0.00	0.00	0.00	6
Rash	0.00	0.00	0.00	6
Electrolyte Imbalance	0.00	0.00	0.00	5
Bleeding / Haemorrhage	0.00	0.00	0.00	3

Precision — of the cases flagged as this reaction, the proportion that truly were. **Recall** (sensitivity) — of the cases that truly were this reaction, the proportion detected. **Support** — how many such cases existed in the test set. Note how small the supports are: with 3 to 13 examples, these estimates would be highly unstable even if the model performed well.

6.5 Unsupported claims in the repository

Two fabricated performance claims are hard-coded into the report generator

`scripts/model_trainer.py` writes the following statements into every report it generates, as fixed text, regardless of the measured results:

- "Pharmacogenomic hypersensitivities ... reach > 92% recall" — the measured recall for hypersensitivity classes is between 0.00 and 0.50.
- "Omitting PGx features reduces weighted F1 from 0.871 to 0.774" — the measured weighted F1 is 0.18, and no ablation producing 0.871 exists.

Neither is supported by any artifact in the repository. They are presented here because identifying this kind of claim is a transferable skill: a stated metric is only as good as the experiment that produced it, and a number that cannot be traced to a reproducible run should not be believed.

6.6 What would be needed to make this work

1. **Real data** with linked outcomes, under appropriate governance.
2. **One consistent label space** between training and evaluation — the current mismatch must be fixed before any figure is meaningful.
3. **Imbalance handling** — class weighting or resampling.
4. **Appropriate metrics** — per-class recall, precision-recall AUC, and calibration, rather than overall accuracy.
5. **Calibration assessment** — a stated 70% risk should correspond to the event occurring in about 70% of such patients.
6. **External validation** on a separate institution's population.
7. **Prospective evaluation** of whether using the tool changes outcomes.

7 Layer 3 — The AI narrative

The third layer uses a large language model to convert the numerical output into clinical prose: an interpretation, monitoring suggestions, and a mitigation plan.

Provider	NVIDIA NIM (OpenAI-compatible endpoint)
Primary model	<code>nvidia/nemotron-3-super-120b-a12b</code>
Fallback model	<code>nvidia/nemotron-3-ultra-550b-a55b</code>
Final fallback	Deterministic rule-based text

Graceful degradation

Language model endpoints fail routinely — they return "temporarily overloaded", or take too long. The system tries the primary model, retries once if it reports itself overloaded, falls through to the secondary model, and finally emits fixed rule-based text. A total time budget bounds the whole sequence so a request cannot hang.

Every response carries an `ai_generated` flag, so it is always possible to tell whether the text was written by the model or by the deterministic fallback.

Fluency is not accuracy

A language model produces text that reads as though written by a confident clinician, whether or not the content is correct. It has no access to the patient beyond the fields supplied, and it cannot verify the risk score it is asked to explain. Here it is explaining a number that Section 6 shows to be uninformative — so a plausible, well-written narrative may be describing a meaningless figure convincingly. Every AI-generated statement requires independent verification.

8 Safety, limitations and governance

Not suitable for clinical use

This system must not inform decisions about any real patient. The limitations below are not caveats around a working tool; several are fundamental.

Model limitations

- **Performs below the majority-class baseline** — 15.5% against 89.5%.
- **Trained entirely on synthetic data** — no real patient contributed.
- **Never externally validated** — no prospective or independent evaluation.
- **Broken evaluation harness** — training and test label spaces do not match.
- **Uncalibrated** — the stated percentages have no verified relationship to real event frequencies.
- **Most ADR classes are never predicted correctly** — precision and recall of zero for the majority of reaction types.

System limitations

- **No authentication** — every endpoint is publicly accessible.
- **No database** — assessments are held in memory and lost on restart.
- **No audit trail** — a clinical requirement that is entirely absent.
- **No regulatory clearance** — a tool of this type would be a regulated medical device in most jurisdictions.

What the project does demonstrate

The engineering is sound even though the model is not. It is a working example of a full clinical decision support architecture: validated data capture, a served ML model, deterministic rule engines, language model integration with sensible failure handling, and a documented deployment. As a teaching artifact for how such systems are built — and how to evaluate the claims they make — it is useful. As a clinical instrument, it is not.

What to take away

- Always ask what baseline an accuracy figure beats.
- Ask whether the training data is real, and whether validation was external.
- For rare outcomes, demand per-class recall, not overall accuracy.
- Check whether a quoted metric can be traced to a reproducible run.
- Treat fluent AI prose as a presentation layer, never as evidence.

9 Glossary

TERM	IN CLINICAL LANGUAGE
Accuracy	Proportion of all predictions that were correct. Misleading when one outcome dominates.
Precision	Positive predictive value — of those flagged, how many truly had the outcome.
Recall / sensitivity	Of those who truly had the outcome, how many were detected.
F1 score	Harmonic mean of precision and recall; a single number balancing the two.
Support	Number of true cases of a class in the test set — the denominator behind the percentages.
Baseline	The score achievable by a trivial rule. A model must beat it to be worth anything.
Class imbalance	One outcome vastly outnumbering others — as with any rare disease.
Calibration	Whether a stated 70% risk corresponds to the event occurring in 70% of such patients.
Training / test split	Holding data back to test on, so performance is not measured on cases already seen.
External validation	Testing on a different population or institution — the standard for real clinical claims.
Gradient boosting	Many simple decision trees in sequence, each correcting its predecessors' errors.
Feature	One input variable, e.g. eGFR or age.
Hyperparameter	A setting chosen before training, e.g. how many trees to build.
Synthetic data	Computer-generated records, not from real patients.
Large language model	A text-generating model. Fluent by construction; not a source of verified fact.
Pharmacogenomics	How inherited genetic variation alters drug response.
CYP enzymes	Cytochrome P450 — the main family metabolising most drugs.
Poor metaboliser	Reduced enzyme activity, so the drug accumulates at standard doses.
HLA allele	Immune system variant; specific alleles predict severe hypersensitivity to particular drugs.

PhenoRx Technical & Methodological Overview · System version 2.1.0. All performance figures in this document were taken from the repository's own artifacts or measured directly from the shipped model; none are estimated. The model evaluated is `models/adr_model.pkl`.

This document describes a research and teaching prototype. It is not a clinical tool, has no regulatory clearance, and must not be used in the care of any patient.